NumPy Basics for Beginners

Here is the Kaggle Version

What is NumPy?

- A Python library for numerical computing
- Provides **multidimensional arrays** (like grids of numbers)
- Faster than regular Python lists for mathematical operations

Real-World Applications

1. Machine Learning (data preprocessing)
2. Data Analysis (statistical calculations)
3. Image Processing (pixel arrays)
4. Physics Simulations (3D calculations)
5. Financial Modeling (stock price analysis)

1. Installation & Setup

```
# If you don't have NumPy:
!pip install numpy

# Import the library
import numpy as np
```

```
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (2.0.2)
```

2. Creating Arrays

```
# From a Python list
numbers = np.array([1, 2, 3, 4])
print("1D Array:", numbers)

# 2D array (matrix)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print("\n2D Array:\n", matrix)

# Special arrays
zeros = np.zeros((2, 3))  # 2x3 grid of zeros
ones = np.ones((3, 2))    # 3x2 grid of ones
range_array = np.arange(0, 10, 2)  # Like range(): 0-8, step 2

print("\nZeros:\n", zeros)
print("\nOnes:\n", ones)
print("\nRange array:", range_array)
```

```
1D Array: [1 2 3 4]

2D Array:
 [[1 2 3]
 [4 5 6]]

Zeros:
 [[0. 0. 0.]
 [0. 0. 0.]]

Ones:
 [[1. 1.]
 [1. 1.]
 [1. 1.]]

Range array: [0 2 4 6 8]
```

Detailed breakdown of `np.zeros()`, `np.ones()`, and `np.arange()` for array initialization in NumPy:

---

1. `np.zeros(shape, dtype=float, order='C')`

- **Purpose**: Creates an array filled with **zeros** of the specified shape and data type.
- **Parameters**:
  - `shape`: Tuple defining the array dimensions (e.g., `(2, 3)` for a 2x3 matrix).
  - `dtype`: Optional (default is `float64`). Specify the data type (e.g., `int`, `float`).
  - `order`: Optional (row-major/C-style or column-major/F-style memory layout).

Example from our code:

```
zeros = np.zeros((2, 3))
# Output:
# [[0. 0. 0.]
#  [0. 0. 0.]]
```

- Creates a **2x3 array** of **floating-point zeros** (default `dtype`).

---

2. `np.ones(shape, dtype=None, order='C')`

- **Purpose**: Creates an array filled with **ones** of the specified shape and data type.
- **Parameters**: Similar to `np.zeros()`.

Example from our code:

```
ones = np.ones((3, 2))
# Output:
# [[1 1]
#  [1 1]
#  [1 1]]
```

- Creates a **3x2 array** of **integers** (defaults to `int` if no `dtype` is specified).

---

3. `np.arange([start], stop, [step], dtype=None)`

- **Purpose**: Generates a **1D array** of evenly spaced values within a range.
- **Parameters**:
  - `start`: Starting value (default: `0`).
  - `stop`: End of interval (the array stops **before** this value).
  - `step`: Increment between values (default: `1`).
  - `dtype`: Data type (inferred from `start`, `stop`, `step` if not provided).

Example from our code:

```
range_array = np.arange(0, 10, 2)
# Output: [0 2 4 6 8]
```

- Starts at `0`, increments by `2`, stops **before** `10`.
- Behaves like Python's `range()`, but returns a **NumPy array** instead of a list.

Key Differences from Vanilla Python

NumPy Function	Python Equivalent
`np.zeros((n,))`	`[0] * n` (for lists)
`np.ones((n,))`	`[1] * n` (for lists)
`np.arange(a, b)`	`range(a, b)`

Why Use These Functions?

- **Efficiency**: NumPy arrays are faster and use less memory than Python lists.
- **Vectorization**: Designed for mathematical operations (e.g., `zeros` for preallocating arrays).
- **Flexibility**: Easily create arrays of any shape/dtype for computations.

Common Use Cases

- `zeros`:
  - Preallocate memory for results in algorithms.
  - Initialize weight matrices in neural networks.
- `ones`:
  - Create default matrices for probability distributions.
  - Initialize counters or placeholders.
- `arange`:
  - Generate sequences for plotting or indices.
  - Create time steps for simulations.

Edge Cases & Tips

- **Data Types**:

```
np.zeros((2, 2), dtype=int)  # 2x2 integers
np.arange(0, 5, 0.5)        # Handles floating-point steps: [0.0, 0.5, 1.0, ..., 4.5]
```

- **1D vs. Multi-Dimensional**:

```
np.zeros(5)       # 1D array of 5 zeros.
np.zeros((3, 3))  # 3x3 2D array.
```

- **Avoiding Floating-Point Zeros**:
  Use `dtype=int` if you want integer zeros (e.g., `np.zeros((2,3), dtype=int)`).

3. Array Properties

```
arr = np.array([[1, 2, 3], [4, 5, 6]])

print("Shape:", arr.shape)  # (rows, columns) → (2, 3)
print("Size:", arr.size)     # Total elements → 6
print("Dimensions:", arr.ndim)  # 2 (2D array)
print("Data type:", arr.dtype)  # int32 or float64
```

```
Shape: (2, 3)
Size: 6
Dimensions: 2
Data type: int64
```

Detailed explanation of the code and the outputs:

Output Explanation

1. `arr.shape`:
   - Returns the **dimensions** of the array as a tuple.
   - Output: `(2, 3)` → 2 rows, 3 columns.
   - Example:

```
# (rows, columns) → (2, 3) for this array.
```

2. `arr.size`:
   - Returns the **total number of elements** in the array.
   - Output: `6` → 2 rows × 3 columns = 6 elements.
3. `arr.ndim`:
   - Returns the **number of dimensions** (axes) of the array.
   - Output: `2` → 2D array (rows and columns).
4. `arr.dtype`:

- Returns the **data type** of the elements in the array.
- Output: `int32` or `int64` (depends on system) → Elements are integers.

Key Concepts

1. `arr.shape`

- **Purpose**: Reveals the array's dimensions.
- **Example**:

```
# 3D array example:
arr_3d = np.array([[[1], [2]], [[3], [4]]])
print(arr_3d.shape) → (2, 2, 1)
```

2. `arr.size`

- **Purpose**: Total elements = product of all dimensions.
- **Formula**:

```
size = arr.shape[0] * arr.shape[1] * ... for all dimensions.
```

3. `arr.ndim`

- **Purpose**: Number of axes (dimensions).
- Examples:
  - 1D: `[1, 2, 3]` → `ndim=1`.
  - 3D: `[[[1]], [[2]]]` → `ndim=3`.

4. `arr.dtype`

- **Purpose**: Shows the data type of elements (e.g., `int32`, `float64`, `bool`).
- **Control with `dtype` parameter**:

```
arr_float = np.array([1, 2, 3], dtype=float) → dtype=float64
arr_bool = np.array([True, False]) → dtype=bool
```

Why These Attributes Matter

- **Shape/Dimensions**: Critical for ensuring arrays are compatible for operations (e.g., matrix multiplication).
- **Data Type**: Ensures correct precision/compatibility (e.g., `float64` for decimals).
- **Size**: Useful for memory management and algorithm optimization.

Common Pitfalls

- **`shape` vs. `size`**:

```
arr.shape → (2, 3)  # Dimensions
arr.size → 6        # Total elements
```

- **Default `dtype`**:
  - Integers → `int32` / `int64`.
  - Floats → `float64`.
  - Strings → `object` (if mixed data types).

4. Basic Operations

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Element-wise operations
print("a + b:", a + b)  # [5 7 9]
print("a * 2:", a * 2)  # [2 4 6]
```

```
# Mathematical functions
print("Square root:", np.sqrt(a))  # [1.  1.414 1.732]
print("Exponential:", np.exp(a))   # [ 2.718  7.389 20.085]
```

```
a + b: [5 7 9]
a * 2: [2 4 6]
Square root: [1.         1.41421356 1.73205081]
Exponential: [ 2.71828183  7.3890561  20.08553692]
```

Detailed explanation of the code and its operations:

Key Concepts

1. Element-Wise Operations

- `a + b`:
  - Adds corresponding elements of `a` and `b`.
  - **Requirement**: Arrays must have **matching shapes** (same dimensions).
  - **Output**: `[5, 7, 9]` (since `[1+4, 2+5, 3+6]`).
- `a * 2`:
  - Multiplies each element in `a` by `2`.
  - **Output**: `[2, 4, 6]`.

---

2. Scalar Multiplication vs. Matrix Multiplication

- **Scalar Multiplication** (`a * 2`):
  - Scales each element by a scalar (single number).
- **Matrix Multiplication**:
  - Requires `np.dot(a, b)` or `@` operator (not shown here).

---

3. Universal Functions (ufuncs)

Functions like `np.sqrt` and `np.exp` apply **element-wise** to arrays:

- `np.sqrt(a)`:
  - Computes √1, √2, √3 → `[1.0, ~1.414, ~1.732]`.
- `np.exp(a)`:
  - Computes e^1, e^2, e^3 → `[2.718..., 7.389..., 20.085...]`.
- **Output Type**:
  - Results are **floats**, even if inputs are integers.

Output Details

1. `a + b`:
   - **Result**: `[5 7 9]` (integer array if inputs are integers).
2. `a * 2`:
   - **Result**: `[2 4 6]`.
3. `np.sqrt(a)`:
   - **Result**: `[1. 1.41421356 1.73205081]`.
4. `np.exp(a)`:
   - **Result**: `[2.71828183 7.3890561 20.08553692]`.

Critical Features

- **Vectorization**:
  - Operations are applied to **all elements simultaneously**, eliminating the need for loops.

- **Broadcasting**:
  - Automatic alignment of arrays for operations (works here since `a` and `b` have the same shape).

Edge Cases & Notes

- **Shape Mismatch**:
  - `a + b` would fail if their shapes differ (unless broadcasting rules apply).
- **Data Type Promotion**:
  - Mixing integers and floats:

    ```
    a_float = a.astype(float)
    a_float + b → floats.
    ```

- **In-Place Operations**:
  - Use `+=`, `*=`, etc., to modify arrays in place.

Why This Matters

- **Efficiency**: Vectorized operations are **~10-100x faster** than Python loops.
- **Readability**: Simplifies code (e.g., `a + b` vs. looping over elements).

5. Aggregation/Statistical Functions

```
temps = np.array([22.5, 23.7, 24.3, 21.8, 19.4])

# methods of the ndarray class
print("Max temperature:", temps.max())  # 24.3
print("Average:", temps.mean())         # 22.34
print("Total sum:", temps.sum())         # 111.7
print("Standard Deviation:", temps.std())  # 1.712
# The median() function is not implemented as a method of the ndarray class. Instead, it is a standalone function in the NumPy
print("Median:", np.median(temps))    # 22.5
```

```
Max temperature: 24.3
Average: 22.339999999999996
Total sum: 111.69999999999999
Standard Deviation: 1.711841114122453
Median: 22.5
```

Detailed Explanation of Aggregation/Statistical Functions

Code Breakdown

```
temps = np.array([22.5, 23.7, 24.3, 21.8, 19.4])
print("Max temperature:", temps.max())        # Maximum value
print("Average:", temps.mean())           # Mean value
print("Total sum:", temps.sum())           # Sum of all elements
print("Standard Deviation:", temps.std())  # Population standard deviation
print("Median:", np.median(temps)          # Median (calculated via numpy's standalone function)
```

Function Explanations

1. `temps.max()`

- **Purpose**: Returns the **maximum value** in the array.
- **Output**: `24.3` (largest value in `temps`).
- **Formula**: `max(22.5, 23.7, 24.3, 21.8, 19.4) = 24.3`.

---

2. `temps.mean()`

- **Purpose**: Computes the **average** of the array.
- **Output**: `22.34` (sum divided by count).

- **Formula**:

$$\text{Mean} = \frac{22.5 + 23.7 + 24.3 + 21.8 + 19.4}{5} = 22.34$$

---

3. `temps.sum()`

- **Purpose**: Sums **all elements** in the array.
- **Output**: `111.7` (22.5 + 23.7 + 24.3 + 21.8 + 19.4 = 111.7).

---

4. `temps.std()`

- **Purpose**: Computes the **standard deviation**, a measure of data spread.
- **Output**: `1.712` (rounded).
- **Formula**:
  - **Population Standard Deviation** (default in `np.std()`):

$$\sigma = \sqrt{\frac{\sum(x_i - \mu)^2}{N}}$$

    - `N = 5` (number of elements).
    - Calculation:
      - Squared deviations sum: ~14.652 (detailed in prior calculations).
      - Variance: $14.652/5 \approx 2.9304$.
      - Standard deviation: $\sqrt{2.9304} \approx 1.712$.
  - To compute **sample standard deviation** (divide by (N-1)), use `temps.std(ddof=1)`.

---

5. `np.median(temps)`

- **Purpose**: Returns the **median** (middle value of sorted data).
- **Output**: `22.5` (sorted array: `[19.4, 21.8, 22.5, 23.7, 24.3]`; middle value is the 3rd element.
- **Why a Standalone Function?**: The median requires sorting the array, which isn't a trivial array attribute like `max()` or `mean()`, so it's implemented as a standalone function.

Key Observations

- **Vectorization**: All operations are applied across the entire array efficiently.
- **Data Type**: Results are **floats**, even if inputs are integers.
- **Axis Parameter**: For multi-dimensional arrays, use `axis=` to specify the dimension (e.g., `axis=0` for column-wise operations in 2D arrays).

Why These Functions Matter

- **Efficiency**: NumPy's C-backend calculations are **~10-100x faster** than pure Python loops.
- **Convenience**: One-line statistical summaries for large datasets.

Common Use Cases

Function	Use Case
`.max()`, `.min()`	Find extremes in data.
`.mean()`, `.sum()`	Compute averages or totals.
`.std()`, `.var()`	Analyze data spread.
`np.median()`	Robust central tendency metric.

Edge Cases & Notes

1. **Standard Deviation**:
   - Use `ddof=1` for sample standard deviation (divided by $N - 1$).
   - Example:

     ```
     sample_std = temps.std(ddof=1) → ~1.84 (for this dataset).
     ```

2. **Median Calculation**:

- For even-length arrays, the median averages the two middle values.

3. **Empty Arrays**:
   - Functions return `nan` or errors if the array is empty.

Why `median` is a Standalone Function

- `np.median()` requires sorting the array and finding the middle value, which is more computationally involved than `.max()` or `.mean()`.

Example Calculations

Let's verify `std()` manually (population SD):

1. **Mean**: 22.34
2. **Squared deviations**:

$$(22.5 - 22.34)^2 = 0.0256$$
$$(23.7 - 22.34)^2 = 1.8496$$
$$(24.3 - 22.34)^2 = 3.8416$$
$$(21.8 - 22.34)^2 = 0.2916$$
$$(19.4 - 22.34)^2 = 8.6436$$

Sum: $0.0256 + 1.8496 + 3.8416 + 0.2916 + 8.6436 = 14.652$

Variance: $14.652/5 \approx 2.9304$

Standard Deviation: $\sqrt{2.9304} \approx 1.712$.

Why Use These Functions?

- **Speed**: NumPy's optimized C-based computations.
- **Simplicity**: One-line statistical summaries.
- **Robustness**: Handle large datasets effortlessly.

Practice Tip

For a 2D array:

```
matrix = np.array([[1, 2], [3, 4]])
print(matrix.sum(axis=0)) → [4, 6]  # Sum along columns
```

6. Reshaping Arrays

```
original = np.arange(1, 7)  # [1 2 3 4 5 6]
reshaped = original.reshape(2, 3)

print("Original shape:", original.shape)  # (6,)
print("Reshaped:\n", reshaped) #[[1 2 3]\n[4 5 6]]
```

```
Original shape: (6,)
Reshaped:
 [[1 2 3]
 [4 5 6]]
```

Detail explanation.

In NumPy, **reshaping arrays** changes the dimensions of an array while keeping its data intact.

Example Breakdown:

1. **Original Array**:
   `original = np.arange(1, 7)` creates a **1D array**: `[1, 2, 3, 4, 5, 6]`.
   Its shape is `(6,)` (6 elements in a single dimension).

2. **Reshaping**:
   `original.reshape(2, 3)` converts this into a **2D array** with:
   - 2 rows (`2`),
   - 3 columns (`3`).

   The total elements must match: `2 × 3 = 6` (same as original length).
3. **Result**:

   ```
   [[1 2 3]
    [4 5 6]]
   ```

   The shape becomes `(2, 3)` (2 rows, 3 columns).

Key Points:

- **Data remains the same**, just arranged differently.
- The new shape must preserve the total number of elements.
- Reshape returns a **new array**; the original remains unchanged.

Why Use It?

Reshaping is used to prepare data for operations requiring specific dimensions (e.g., matrices for math operations or input for ML models).

7. Real-World Example: Image Processing

```
# Simulate RGB image (3 color channels)
image = np.random.randint(0, 256, (128, 128, 3), dtype=np.uint8)
print("Image shape (height, width, channels):", image.shape)

# Convert to grayscale (average RGB values)
grayscale = image.mean(axis=2)
print("Grayscale shape:", grayscale.shape)
```

```
Image shape (height, width, channels): (128, 128, 3)
Grayscale shape: (128, 128)
```

Here's a detailed explanation of the image processing example:

Example Breakdown:

1. **Simulated RGB Image**

   ```
   image = np.random.randint(0, 256, (128, 128, 3), dtype=np.uint8)
   ```

   - Creates a **3D array** simulating an RGB image:
     - `128x128` pixels (height × width)
     - `3` color channels (Red, Green, Blue)
   - Each pixel has 3 values between 0–255 (standard 8-bit color depth).
   - Shape: `(128, 128, 3)`
2. **Grayscale Conversion**

   ```
   grayscale = image.mean(axis=2)
   ```

   - `axis=2` targets the **3rd dimension** (color channels).
   - Computes the **average of RGB values** for each pixel:

     ```
     grayscale_value = (R + G + B) / 3
     ```

   - Removes the color channel dimension (from 3D → 2D).
   - New shape: `(128, 128)`

3. **Output**:

```
Original shape: (128, 128, 3)
Grayscale shape: (128, 128)
```

Key Concepts:

1. **Axis Parameter**:
   - `axis=2` operates along the 3rd dimension (color channels).
   - Think of it as "collapsing" the color dimension by averaging.
2. **Dimensionality Reduction**:
   - Original: 3D (height, width, channels)
   - After `.mean(axis=2)`: 2D (height, width)
3. **Grayscale Math**:
   - Each pixel's brightness is computed from its RGB values.
   - Other methods (like luminance weighting) exist, but averaging is simplest here.

Why This Matters:

- **Image Processing**:
  Reduces color information to simplify analysis (e.g., edge detection, compression).
- **Machine Learning**:
  Converts color images to grayscale to reduce computational load.
- **Data Analysis**:
  Demonstrates manipulating multidimensional data along specific axes.

Common Mistakes to Avoid:

- **Shape Mismatch**:
  Ensure the operation aligns with your data's dimensions.
- **Axis Confusion**:
  Double-check which axis you're modifying (e.g., `axis=0` would affect rows, not channels).

8. Real-World Example: Stock Prices

```
# Simulate 5 days of stock prices for 3 companies
stock_prices = np.array([
    [150, 275, 42],  # Day 1
    [152, 274, 41],  # Day 2
    [153, 278, 40],  # Day 3
    [151, 280, 39],  # Day 4
    [155, 282, 38],  # Day 5
])

# Calculate daily changes
daily_changes = np.diff(stock_prices, axis=0)
print("Daily price changes:\n", daily_changes)   #[[ 2 -1 -1][ 1  4 -1][-2  2 -1][ 4  2 -1]]
```

```
Daily price changes:
 [[ 2 -1 -1]
 [ 1  4 -1]
 [-2  2 -1]
 [ 4  2 -1]]
```

Here's a detailed breakdown of the **stock prices example** using NumPy:

Example Breakdown

1. Original Data

```
stock_prices = np.array([
    [150, 275, 42],  # Day 1
    [152, 274, 41],  # Day 2
    [153, 278, 40],  # Day 3
    [151, 280, 39],  # Day 4
    [155, 282, 38],  # Day 5
])
```

- **Shape**: `(5, 3)` (5 days × 3 companies).
- **Structure**:
  - Rows = Days (Day 1 to Day 5).
  - Columns = Companies (e.g., Company A, B, C).

---

2. Calculating Daily Changes

```
daily_changes = np.diff(stock_prices, axis=0)
```

- `np.diff` computes the **difference between consecutive elements** along a specified axis.
- `axis=0` means:
  - Compare **rows** (days) for each company.
  - Compute `Day 2 - Day 1`, `Day 3 - Day 2`, etc.

---

3. Result

```
daily_changes.shape → (4, 3)
daily_changes = [
  [ 2, -1, -1],  # Day 2 - Day 1
  [ 1,  4, -1],  # Day 3 - Day 2
  [-2,  2, -1],  # Day 4 - Day 3
  [ 4,  2, -1]   # Day 5 - Day 4
]
```

- **Shape reduction**: The result has **1 fewer row** (5 → 4) because differences are between consecutive days.
- **Interpretation**:
  - Each entry shows **daily price change** for a company.
  - Example:
    - Company 1 (first column):
      - Day 2: `152 - 150 = +2` (up).
      - Day 5: `155 - 151 = +4` (up.

Key Concepts

1. `np.diff(axis=0)`:
   - Subtracts each row from the next row along the vertical axis.
   - Formula: `daily_changes[i] = stock_prices[i+1] - stock_prices[i]`.
2. **Axis Parameter**:
   - `axis=0` → Operate **row-wise** (compare days).
   - `axis=1` (if used) would compare columns (companies) within the same day, which isn't useful here.
3. **Dimensionality**:
   - Input shape: `(5, 3)` → Output shape: `(4, 3)` (reduced by 1 in the specified axis.

Why This Matters

- **Financial Analysis**:
  - Tracks how stock prices move **day-over-day**.
  - Helps identify trends (e.g., rising/falling stocks.

- **Data Preparation**:
    - Converts raw prices into **changes**, which is useful for analysis (e.g., volatility calculation.

Common Questions

Q: What if `axis=1` is used instead?

- Output shape would be `(5, 2)` (columns reduced by 1.
- This would compute differences **within each day** (e.g., compare Company B to A, Company C to B for the same day. Not meaningful for this use case.

Q: How to calculate percentage change instead of absolute?

```
daily_percent_change = (np.diff(stock_prices, axis=0) / stock_prices[:-1]) * 100
```

- Uses `stock_prices[:-1]` to avoid division by zero (since `diff` reduces the first dimension.

Visualization

Original Prices (5 days × 3 companies)	Daily Changes (4 days × 3 companies)
150	275
152	274
153	278
151	280
155	282

Practice Exercise

Try:

```
# Calculate weekly average price
weekly_avg = stock_prices.mean(axis=0)  # Averages across days (rows.
print("Average price per company:", weekly_avg)
```

This uses `axis=0` to collapse days into an average for each company.

My Next Notebook we will be discussing how to use matplotlib with numpy

Notes

When to Use NumPy + Matplotlib

Task	NumPy	Matplotlib
Processing data	✅ Array operations (e.g., `.mean(axis=...)`	❌
Visualizing trends	❌	✅ Plots, heatmaps, etc.

Summary

- **NumPy** handles **data manipulation** (reshaping, calculations).
- **Matplotlib** handles **data visualization** (plots, heatmaps.
- Combine them to analyze and present data effectively.

Upcoming Topics: Using NumPy with Matplotlib

In the next notebook, we will delve deeper into how NumPy and Matplotlib can be used together to create powerful visualizations for real-world data analysis. We will build upon the concepts and examples discussed in this notebook to explore more advanced plotting techniques and data visualization methods.

Recap of Real-World Examples

Real-World Example: Image Processing

- **Objective**: Convert an RGB image to a grayscale image.

- **Process**:
  - Generated a random RGB image using NumPy.
  - Converted the RGB image to grayscale by computing the mean of the color channels.
  - Displayed both the original RGB image and the grayscale image using Matplotlib.
- **Key Concepts**:
  - NumPy for generating and manipulating image data.
  - Matplotlib for displaying images and customizing plots.

Real-World Example: Stock Prices

- **Objective**: Analyze and visualize stock prices and daily changes.
- **Process**:
  - Created a NumPy array containing stock prices for multiple companies over several days.
  - Calculated daily changes using `np.diff`.
  - Plotted the stock prices over time and the daily changes with colored bars indicating positive (green) or negative (red) changes.
- **Key Concepts**:
  - NumPy for numerical operations and data manipulation.
  - Matplotlib for creating line plots and bar charts with custom colors and labels.

What's Next?

In the next notebook, we will explore more advanced topics, including:

- Creating complex visualizations with multiple subplots.
- Using different types of plots (e.g., histograms, scatter plots).
- Customizing plots with annotations, legends, and color maps.
- Integrating NumPy and Matplotlib for real-world data analysis.

Stay tuned for more insights and practical examples on how to leverage NumPy and Matplotlib for data visualization!

Real-World Example: Image Processing

```
import numpy as np
import matplotlib.pyplot as plt

# Simulate an RGB image (3D array: height x width x channels)
rgb_image = np.random.randint(0, 256, (100, 100, 3), dtype=np.uint8)

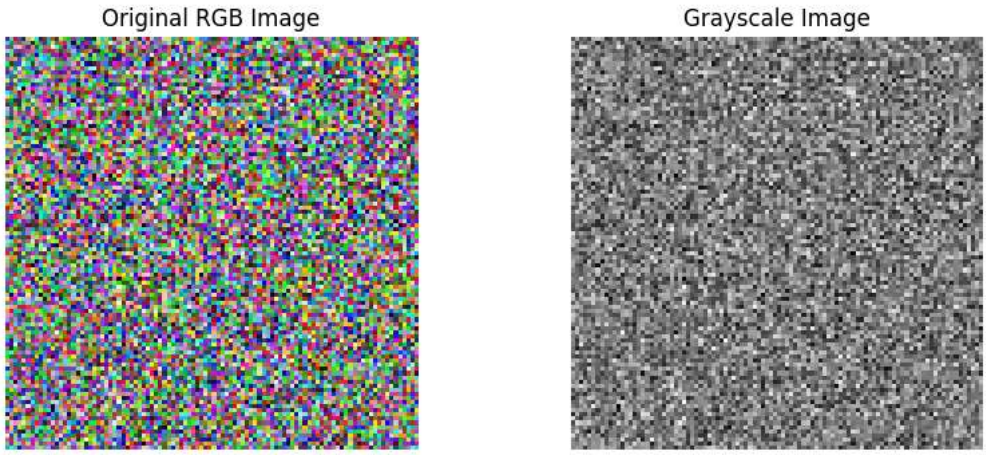
# Convert to grayscale (2D array: height x width)
# Use np.mean to compute the mean and then convert the result to uint8
grayscale = np.mean(rgb_image, axis=2).astype(np.uint8)

# Display using Matplotlib
plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.imshow(rgb_image)
plt.title("Original RGB Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(grayscale, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.show()
```



Real-World Example: Stock Prices

```
import numpy as np
import matplotlib.pyplot as plt

# Stock Prices Example
stock_prices = np.array([
    [150, 275, 42],
    [152, 274, 41],
    [153, 278, 40],
    [151, 280, 39],
    [155, 282, 38],
])

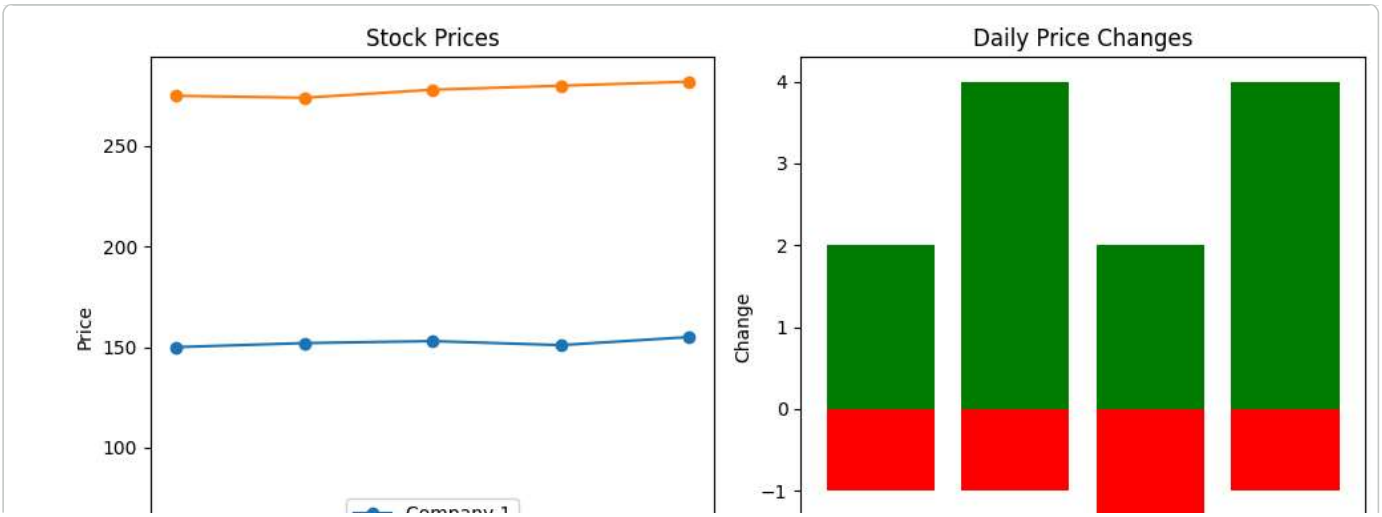
# Calculate daily changes
daily_changes = np.diff(stock_prices, axis=0)

# Plot prices
plt.figure(figsize=(10, 5))

# Prices over time
plt.subplot(1, 2, 1)
for i in range(stock_prices.shape[1]):
    plt.plot(stock_prices[:, i], marker="o", label=f"Company {i+1}")
plt.title("Stock Prices")
plt.xlabel("Day")
plt.ylabel("Price")
plt.legend()

# Daily changes
plt.subplot(1, 2, 2)
for i in range(daily_changes.shape[1]):
    # Color bars green/red based on change
    colors = ["g" if change > 0 else "r" for change in daily_changes[:, i]]
    plt.bar(np.arange(len(daily_changes)), daily_changes[:, i], color=colors)
plt.title("Daily Price Changes")
plt.xlabel("Day")
plt.ylabel("Change")
plt.xticks(ticks=np.arange(len(daily_changes)), labels=["Day 2-1", "Day 3-2", "Day 4-3", "Day 5-4"])

plt.tight_layout()
plt.show()
```


Stock Prices
Price
250
200
150
100
Daily Price Changes
Change
4
3
2
1
0
−1